PitchReady

Technical Architecture Report

System design, data models, API contracts, agent pipeline, deployment topology · March 2026

1. System Overview

PitchReady is a multi-agent AI web application that stress-tests startup pitch decks against domain-expert knowledge in a specific vertical. It accepts PDF or PPTX uploads, orchestrates a four-agent Claude API pipeline via LangGraph, and returns a structured readiness report covering assumption gaps, blind spots, and sharpening prompts.

The system is composed of five independently deployable concerns: a Next.js frontend on Vercel, a FastAPI backend on a DigitalOcean Droplet, a LangGraph worker process co-located on the same Droplet, a Neon serverless Postgres database, and Upstash Redis as the async job queue.

2. High-Level Architecture

The architecture follows a producer-consumer pattern. The FastAPI backend acts as the producer — it validates requests, stores metadata, and enqueues jobs. The LangGraph worker is the consumer — it runs the AI pipeline and writes results back to Postgres.

```
Browser (Next.js / Vercel)
      |
      |  HTTPS  (REST + SSE)
      v
DigitalOcean Droplet
+-----------------------------------------+
|  Nginx  (reverse proxy + SSL termination) |
|        |                                 |
|        v                                 |
|  FastAPI  (Python 3.11)                  |
|   - Auth middleware (Neon JWT verify)      |
|   - File pre-sign  (DO Spaces)             |
|   - Job enqueue    (Upstash Redis)         |
|   - SSE status stream                      |
|        |                                 |
|        v                                 |
|  Upstash Redis  (job queue)                |
|        |                                 |
|        v                                 |
|  LangGraph Worker  (Python 3.11)           |
|   - Agent 1: Extractor                     |
|   - Agent 2: Assumption Mapper             |
|   - Agent 3: Domain Interrogator           |
|   - Agent 4: Report Synthesizer            |
```

```
 |        |                                  |
 +--------|----------------------------------+
          |
    +-----+-----+
    |           |
 Neon DB    DO Spaces
(Postgres) (PDF/PPTX)
    |
 Anthropic
 Claude API
```

3. Frontend — Next.js on Vercel

3.1 Technology Choices

Technology	Version	Purpose
Next.js	14 (App Router)	SSR, routing, API routes
TypeScript	5.x	Type safety across all components
Tailwind CSS	3.x	Utility-first styling
Neon Auth SDK	@neondatabase/auth	JWT-based session management
PostHog JS	posthog-js	Event tracking, session replay
React Query	tanstack/query v5	Server state, polling, caching

3.2 Page Structure

Route	Component	Description
/	LandingPage	Marketing, CTA, pricing
/login	AuthPage	Neon Auth sign-in / sign-up
/dashboard	Dashboard	User's analysis history
/analyze	AnalyzePage	Upload deck, select vertical, submit
/report/[id]	ReportPage	Rendered readiness report
/admin	AdminDashboard	Protected — metrics, pipeline health

3.3 File Upload Flow

Files are never routed through the FastAPI server. The frontend requests a pre-signed PUT URL from the backend, uploads directly to DO Spaces from the browser, then sends only the resulting Spaces object key to the backend for analysis.

```
1. POST /api/upload/presign  { filename, content_type }
   <- { presigned_url, object_key, expires_in }

2. Browser PUT presigned_url  (file bytes direct to DO Spaces)
   <- 200 OK

3. POST /api/analyze  { object_key, vertical }
   <- { job_id }

4. GET /api/jobs/{job_id}/stream  (SSE)
   <- event: agent_complete  data: { agent: 1, latency_ms: 1240 }
   <- event: agent_complete  data: { agent: 2, latency_ms: 980  }
   <- event: done            data: { report_id: 'rpt_abc123'     }
```

4. Backend — FastAPI

4.1 Technology Choices

Technology	Version	Purpose
FastAPI	0.111.x	Async REST API framework
Python	3.11	Runtime
Pydantic v2	2.x	Request/response validation
asyncpg	0.29.x	Async Postgres driver for Neon
boto3	1.34.x	DO Spaces (S3-compatible) SDK
upstash-redis	1.x	Async Redis client
python-jose	3.x	JWT verification for Neon Auth tokens

4.2 API Contract

Method	Endpoint	Auth	Description
POST	/auth/session	None	Exchange Neon token for session
POST	/upload/presign	Required	Get pre-signed DO Spaces URL
POST	/analyze	Required	Enqueue analysis job
GET	/jobs/{id}/stream	Required	SSE stream for job progress
GET	/reports	Required	List user's past reports
GET	/reports/{id}	Required	Fetch full report JSON
GET	/admin/stats	Admin only	Business + pipeline metrics
GET	/admin/runs	Admin only	Per-run cost and latency log

4.3 Middleware Stack

All requests pass through the following middleware in order:

1. CORS — whitelists Vercel deployment domain and localhost:3000
2. Request ID — injects X-Request-ID header for tracing
3. Auth — verifies Neon JWT on all protected routes
4. Rate Limiter — 10 requests/minute per user using Upstash sliding window

5. Usage Guard — blocks analysis requests if user has exhausted free tier and has no active subscription

5. Database Schema — Neon Postgres

5.1 Users Table

```
CREATE TABLE users (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  neon_user_id  TEXT UNIQUE NOT NULL,
  email         TEXT UNIQUE NOT NULL,
  plan          TEXT NOT NULL DEFAULT 'free',  -- free | pro
  runs_used     INTEGER NOT NULL DEFAULT 0,
  runs_limit    INTEGER NOT NULL DEFAULT 3,
  created_at    TIMESTAMPTZ DEFAULT NOW(),
  last_active   TIMESTAMPTZ DEFAULT NOW()
);
```

5.2 Analysis Runs Table

```
CREATE TABLE analysis_runs (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID REFERENCES users(id),
  vertical          TEXT NOT NULL,
  spaces_object_key TEXT NOT NULL,
  status            TEXT NOT NULL DEFAULT 'queued',
                    -- queued | running | done | failed
  agent_1_ms        INTEGER,
  agent_2_ms        INTEGER,
  agent_3_ms        INTEGER,
  agent_4_ms        INTEGER,
  total_ms          INTEGER,
  input_tokens      INTEGER,
  output_tokens     INTEGER,
  cost_usd          NUMERIC(8,4),
  error_message     TEXT,
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  completed_at      TIMESTAMPTZ
);
```

5.3 Reports Table

```
CREATE TABLE reports (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id          UUID REFERENCES analysis_runs(id),
  user_id         UUID REFERENCES users(id),
  vertical        TEXT NOT NULL,
  assumption_map  JSONB NOT NULL,
  blind_spots     JSONB NOT NULL,
  sharpening      JSONB NOT NULL,
  raw_output      TEXT,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

5.4 Indexes

```
CREATE INDEX idx_runs_user_id    ON analysis_runs(user_id);
CREATE INDEX idx_runs_status     ON analysis_runs(status);
CREATE INDEX idx_runs_created    ON analysis_runs(created_at DESC);
CREATE INDEX idx_reports_user_id ON reports(user_id);
CREATE INDEX idx_reports_run_id  ON reports(run_id);
```

6. AI Agent Pipeline — LangGraph

The pipeline is a directed acyclic graph (DAG) of four nodes. LangGraph manages state passing between nodes, retries on transient API failures, and exposes the current node name for SSE progress reporting back to the frontend.

6.1 State Object

A single typed state object is passed through all four agents, accumulating outputs:

```
class PipelineState(TypedDict):
  # Inputs
  raw_text      : str        # Extracted text from PDF/PPTX
  vertical      : str        # e.g. 'logistics_saas'
  run_id        : str

  # Agent outputs
  structured    : dict       # Agent 1 output
  assumptions   : list[dict] # Agent 2 output
  hard_questions: list[str]  # Agent 3 output
  report        : dict       # Agent 4 output

  # Telemetry
  token_counts  : dict[str, int]
  latencies_ms  : dict[str, int]
```

6.2 Agent 1 — Extractor

Parses raw deck text into a structured JSON object. This output is the input to all subsequent agents.

Property	Detail
Model	claude-sonnet-4-5
Input	Raw deck text + extraction system prompt
Output	JSON: { problem, solution, market_size, competitors[], gtm, team, financials }
Avg tokens	~3,800 in / ~800 out
Avg latency	3–5 seconds
Retry policy	2 retries with 2s backoff on 529 / 500 errors

6.3 Agent 2 — Assumption Mapper

Critically analyzes the structured pitch and identifies every unproven, assumed, or hand-wavy claim. Outputs a ranked list by risk level.

Property	Detail
Model	claude-sonnet-4-5
Input	Structured JSON from Agent 1 + assumption analysis prompt
Output	List of { assumption, category, risk_level: high|med|low, why_unproven }
Avg tokens	~2,600 in / ~600 out
Avg latency	2–4 seconds
Retry policy	2 retries with 2s backoff

6.4 Agent 3 — Domain Interrogator

The core moat. This agent is injected with a vertical-specific knowledge base — competitor landscapes, buyer behavior, regulatory context, common failure modes — and generates the hardest questions a domain expert investor would ask.

Property	Detail
Model	claude-sonnet-4-5
Input	Structured JSON + assumptions + vertical KB (injected as system prompt context)
Output	List of { question, why_it_matters, what_good_looks_like }
Avg tokens	~5,000–9,000 in (KB size dependent) / ~1,000 out
Avg latency	5–10 seconds (largest input)
Vertical KB	Markdown file per vertical, loaded from disk at worker startup
Retry policy	2 retries with 3s backoff

Prompt caching opportunity: the vertical KB system prompt is identical across runs in the same vertical. Anthropic prompt caching can reduce Agent 3 input cost by up to 90% once enabled.

6.5 Agent 4 — Report Synthesizer

Combines all prior outputs into the final structured readiness report with three named sections.

Property	Detail
Model	claude-sonnet-4-5
Input	All prior agent outputs + synthesis prompt

Property	Detail
Output	{ assumption_map: [], blind_spots: [], sharpening_prompts: [] }
Avg tokens	~4,500 in / ~1,500 out
Avg latency	4–7 seconds
Retry policy	2 retries with 2s backoff

6.6 LangGraph DAG Definition

```
from langgraph.graph import StateGraph, END

graph = StateGraph(PipelineState)
graph.add_node('extractor',    run_extractor)
graph.add_node('mapper',       run_assumption_mapper)
graph.add_node('interrogator', run_domain_interrogator)
graph.add_node('synthesizer',  run_report_synthesizer)

graph.set_entry_point('extractor')
graph.add_edge('extractor',    'mapper')
graph.add_edge('mapper',       'interrogator')
graph.add_edge('interrogator', 'synthesizer')
graph.add_edge('synthesizer',  END)

pipeline = graph.compile()
```

7. Async Job Queue — Upstash Redis

Analysis runs are processed asynchronously. The FastAPI backend never blocks on AI calls — it enqueues a job and returns a job ID instantly. The LangGraph worker polls the queue and processes one job at a time.

7.1 Job Lifecycle

```
QUEUED  ->  RUNNING  ->  DONE
                  \->  FAILED  (stored with error_message)
```

7.2 Job Payload Schema

```
{
  'job_id'      : 'uuid',
  'run_id'      : 'uuid',
  'user_id'     : 'uuid',
  'object_key'  : 'spaces/uploads/deck_abc.pdf',
  'vertical'    : 'logistics_saas',
  'enqueued_at' : '2026-03-05T10:00:00Z'
}
```

7.3 Worker Loop

```
while True:
  job = await redis.brpop('pitchready:jobs', timeout=5)
  if not job:
      continue

  payload = json.loads(job[1])
  await update_run_status(payload['run_id'], 'running')

  try:
      result = await pipeline.ainvoke(initial_state(payload))
      await save_report(result)
      await update_run_status(payload['run_id'], 'done')
  except Exception as e:
      await update_run_status(payload['run_id'], 'failed', str(e))
```

8. Authentication — Neon Auth

Authentication is handled entirely by Neon Auth. The frontend uses the Neon Auth SDK to manage sign-up, login, and session state. The backend validates JWTs on every protected request using Neon's public JWKS endpoint.

8.1 Auth Flow

6. User signs up or logs in via Neon Auth hosted UI or SDK component
7. Neon issues a signed JWT with user ID and email as claims
8. Frontend stores JWT in memory (not localStorage) and attaches it as Bearer token on all API requests
9. FastAPI middleware fetches Neon's JWKS endpoint (cached for 1 hour) and verifies the JWT signature and expiry
10. Verified user ID is injected into the request context for all downstream handlers

8.2 Admin Route Protection

Admin routes check for a specific user ID or email domain stored in an environment variable. No separate admin user table is needed at MVP stage.

```
ADMIN_USER_IDS=uuid1,uuid2   # set in .env on the Droplet
```

9. File Storage — DigitalOcean Spaces

9.1 Bucket Structure

```
pitchready-uploads/
  uploads/
    {user_id}/
      {run_id}_{original_filename}.pdf
      {run_id}_{original_filename}.pptx
```

9.2 Pre-signed URL Flow

Pre-signed PUT URLs expire after 15 minutes. The worker downloads files using a pre-signed GET URL generated at job execution time, not stored permanently.

Operation	Who Does It	URL Expiry
Upload (PUT)	Browser → Spaces directly	15 minutes
Download (GET)	LangGraph worker	5 minutes (generated per job)
Delete (DEL)	Scheduled cleanup task (90 days)	N/A — server-side boto3 call

10. Deployment Topology

10.1 Docker Compose Services

```
services:
  nginx:
    image: nginx:alpine
    ports: ['80:80', '443:443']
    volumes: [./nginx.conf:/etc/nginx/nginx.conf,
              ./certs:/etc/letsencrypt]
    depends_on: [backend]

  backend:
    build: ./backend
    env_file: .env
    expose: ['8000']
    depends_on: []  # Neon + Upstash are external

  worker:
    build: ./worker
    env_file: .env
    restart: unless-stopped
    # Shares same image as backend, different CMD

  # No postgres / redis containers — both are managed externally
```

10.2 Nginx Configuration Summary

- All HTTP traffic redirected to HTTPS
- SSL certificates managed by Certbot (Let's Encrypt), auto-renewed via cron
- /api/* proxied to FastAPI on port 8000
- /* proxied to Vercel (frontend) — optional, only if using custom domain without Vercel's own DNS
- Gzip compression enabled for JSON responses
- Rate limiting at Nginx level as a secondary guard (primary is FastAPI middleware)

10.3 Environment Variables

Variable	Service	Description
ANTHROPIC_API_KEY	Worker	Claude API authentication
NEON_DATABASE_URL	Backend + Worker	Postgres connection string
NEON_JWKS_URL	Backend	JWT verification endpoint
UPSTASH_REDIS_URL	Backend + Worker	Redis connection string
UPSTASH_REDIS_TOKEN	Backend + Worker	Upstash auth token
DO_SPACES_KEY	Backend + Worker	Spaces access key ID

Variable	Service	Description
DO_SPACES_SECRET	Backend + Worker	Spaces secret access key
DO_SPACES_BUCKET	Backend + Worker	Bucket name
DO_SPACES_REGION	Backend + Worker	e.g. nyc3
ADMIN_USER_IDS	Backend	Comma-separated admin UUIDs
POSTHOG_API_KEY	Frontend (Vercel env)	Analytics ingestion key

11. Admin Dashboard

The admin dashboard is a protected route inside the Next.js app (/admin). It queries the FastAPI admin endpoints directly and renders three panels: business overview, pipeline health, and cost tracker.

11.1 Data Sources

Panel	Data Source	Refresh
User Activity	PostHog (embedded widget or API)	Real-time
Pipeline Health	Neon — analysis_runs table	60s polling
Cost Tracker	Neon — analysis_runs (token + cost cols)	60s polling
Infra Health	DO Monitoring (external link)	DO-managed

11.2 Key Metrics Surfaced

- Total registered users, active last 7 / 30 days
- Total analysis runs, success rate, failure rate by error type
- Per-agent average latency — identifies pipeline bottlenecks
- Total Claude API spend to date, average cost per run, cost per user
- Daily run volume trend (last 30 days)
- Free tier utilization — users at 0 / 1 / 2 / 3 runs used

12. Security Considerations

Concern	Mitigation
JWT forgery	Verified against Neon JWKS — asymmetric RS256 signature
File access control	Pre-signed URLs scoped per user_id path — users cannot access other users' files
API abuse	Rate limiter (10 req/min) + usage guard blocks over-consumption
Prompt injection via deck	Agent 1 extracts to structured JSON first — raw text never injected directly into domain agent prompts
Secret exposure	All secrets in .env on Droplet, never committed to git
Admin route access	UUID whitelist check on all /admin routes + JWT required
CORS	Strict origin whitelist — only Vercel deployment domain allowed

13. Recommended Build Order

Ordered by value delivered per hour of effort. Do not build the next step until the current one is working end-to-end.

#	Milestone	What Gets Built	Est. Effort
1	Auth + skeleton	Neon Auth, login page, JWT middleware	4–6 hrs
2	File upload	DO Spaces, pre-sign endpoint, upload UI	4–5 hrs
3	Extractor agent	Agent 1 synchronously, display parsed JSON	3–4 hrs
4	Full pipeline	Agents 2–4, LangGraph DAG, first real report	8–12 hrs
5	Async queue	Upstash job queue, worker process, SSE stream	6–8 hrs
6	Report UI	Polished report rendering, dashboard history	5–7 hrs
7	Admin dashboard	Pipeline health, cost tracker panels	4–6 hrs
8	Payments	Paywall, subscription, run limit enforcement	6–8 hrs
	Total estimate	**MVP end-to-end**	**40–56 hrs**